

1. Implementa uma lista thread-safe no estilo Copy-On-Write, permitindo leituras concorrentes sem bloquear.
2. O objetivo final é armazenar elementos com segurança quando há threads leitoras e escritoras simultâneas.
3. Em `add()`, usa `ReentrantLock` para exclusão mútua durante escrita.
4. A escrita copia a lista atual, adiciona o item na cópia e substitui a referência `inner` pela nova versão.
5. O campo `volatile` garante visibilidade imediata da nova lista para outras threads.
6. Métodos `get()` e `size()` apenas leem a referência atual, sem lock, tornando leitura rápida.
7. `main()` cria uma thread escritora e outra leitora para demonstrar acesso concorrente.
8. Complexidade: `add()` custa $O(n)$ tempo e $O(n)$ memória; `get()/size()` tendem a $O(1)$.
9. Conceitos usados: generics, threads, lambda expressions, locks e modelo de memória Java.
10. Limitação: ruim para muitas escritas, pois copiar toda a lista frequentemente é caro.